

使用 ADK 和 MCP 建構 Google Workspace AI 代理

來源：<https://codelabs.developers.google.com/google-workspace-mcp-adk?hl=zh-tw>

步驟數：8

使用 ADK 建構專業程式碼 AI 代理，並直接連線至 Google Workspace MCP 伺服器。

目錄

1. [簡介](#)
2. [事前準備](#)
3. [設定 OAuth 同意畫面和用戶端](#)
4. [代理程式開發與測試](#)
5. [正式環境部署](#)
6. [在 Gemini Enterprise 中註冊](#)
7. [清除所用資源](#)
8. [恭喜](#)

1. 簡介

在本程式碼實驗室中，您將瞭解如何使用 **Agent Development Kit (ADK)** 建構專業程式碼 AI 代理，直接連線至官方 **Google Workspace MCP 伺服器**。

Model Context Protocol (MCP) 是一項開放標準，可讓 AI 模型安全地使用遠端伺服器提供的工具。ADK 是 Google 的專業程式碼架構，可用於建構自主代理。結合這些資料，即可根據 Gmail、Google 雲端硬碟、Google 日曆、Google Chat 和聯絡人資料，建立高度客製化的代理程式。

如需可用工具和設定的完整說明文件，請參閱「[設定 Google Workspace MCP 伺服器](#)」。

學習內容

- 在 Google Cloud 雲端專案中啟用 **Google Workspace MCP 服務**。
- 設定 OAuth 同意畫面，並為本機測試和正式版部署作業建立憑證。
- 建立本機 ADK 代理，使用靜態權杖連線至 5 個 Workspace MCP 伺服器。
- 建構及部署適合用於實際工作環境的 ADK 代理，並透過動態權杖插入功能，將代理部署至 **Gemini Enterprise Agent Platform (GEAP) Agent Runtime**。
- 在 **Gemini Enterprise** 中註冊及測試自訂代理。

軟硬體需求

- 網路瀏覽器 (例如 Chrome)。
 - 已啟用計費功能的 Google Cloud 專案。
 - Google Workspace 帳戶 (已開啟智慧功能)。
 - 在本機電腦上安裝 Python 3.11 以上版本。
 - 已安裝並初始化 Google Cloud CLI (`gcloud`)。
-

2. 事前準備

Google Workspace 開發人員預先體驗計畫：加入公開開發人員預先體驗計畫，即可存取 Google Workspace MCP 伺服器。如果尚未註冊，請前往 <https://developers.google.com/workspace/preview> 註冊，並確認帳戶已獲准加入計畫。

建立或選取 Google Cloud 專案

主控台選項

在 [Google Cloud 控制台](#) 中，選取或建立 Google Cloud 專案。

CLI 選項

建立新專案並設為有效專案：

□□□□

```
gcloud projects create YOUR_PROJECT_ID
gcloud config set project YOUR_PROJECT_ID
```

啟用 API

如要使用 Google Workspace MCP 伺服器，您必須同時啟用標準 Google Workspace API 和專屬 MCP 服務。

主控台選項

點選下列按鈕，透過網路瀏覽器啟用這些功能：

CLI 選項：執行下列終端機指令：

```
# Enable standard GWS APIs
gcloud services enable chat.googleapis.com \
  drive.googleapis.com \
  calendar-json.googleapis.com \
  people.googleapis.com \
  gmail.googleapis.com
```

```
# Enable dedicated MCP services
gcloud services enable calendarmcp.googleapis.com \
  chatmcp.googleapis.com \
  drivemcp.googleapis.com \
  gmailmcp.googleapis.com
```

設定 Chat 應用程式

如要使用 Google Chat MCP 伺服器，您必須在 Google Cloud 專案中設定 Chat 應用程式。

1. 在 Google Cloud 控制台中搜尋「Google Chat API」，然後依序點選「Google Chat API」 > 「管理」 > 「設定」。
 2. 設定 Chat 應用程式：
 - 應用程式名稱：`ADK Workspace Agent`
 - 大頭貼網址：`https://developers.google.com/chat/images/quickstart-app-avatar.png`
 - 說明：`MCP server connection for ADK Agent`
 - 在「功能」下方，關閉「啟用互動功能」。
 - 在「記錄」下方，選取「將錯誤記錄至 Logging」。
 3. 按一下 [儲存]。
-

3. 設定 OAuth 同意畫面和用戶端

Google Workspace MCP 伺服器會使用 OAuth 2.0 進行安全驗證。您必須設定 OAuth 同意畫面，並建立兩個不同的用戶端 ID：一個用於本機開發，另一個用於已部署的代理程式。

設定 OAuth 同意畫面

1. 在 Google Cloud 控制台中，按一下左上角的「導覽選單」（漢堡圖示）。依序選取「Google Auth Platform」 > 「Branding」（如果沒有看到 Google Auth Platform，請依序選取「APIs & Services」 > 「OAuth 同意畫面」）。
2. 設定「應用程式資訊」：
 - 應用程式名稱：Workspace ADK Agent
 - 使用者支援電子郵件地址：選取您的電子郵件地址。
3. 設定「目標對象」：選取「內部」。
4. 設定「聯絡資訊」：輸入電子郵件地址，然後點選「建立」。
5. 依序前往「資料存取」 > 「新增或移除範圍」。在「手動新增範圍」下方，新增下列範圍，授予所有 5 項服務的存取權：

```
https://www.googleapis.com/auth/calendar.calendarlist.readonly
https://www.googleapis.com/auth/calendar.events
https://www.googleapis.com/auth/calendar.calendars
https://www.googleapis.com/auth/chat.spaces
https://www.googleapis.com/auth/chat.messages
https://www.googleapis.com/auth/drive.readonly
https://www.googleapis.com/auth/drive.file
https://www.googleapis.com/auth/gmail.readonly
https://www.googleapis.com/auth/gmail.compose
https://www.googleapis.com/auth/gmail.send
https://www.googleapis.com/auth/directory.readonly
https://www.googleapis.com/auth/userinfo.profile
https://www.googleapis.com/auth/contacts.readonly
https://www.googleapis.com/auth/cloud-platform
```

6. 依序點選「新增至表格」、「更新」和「儲存」。

為本機開發作業建立用戶端 ID (電腦應用程式)

1. 在 Google Cloud 控制台中，依序前往「Google Auth Platform」 > 「Clients」（或「APIs & Services」 > 「Credentials」）。按一下「建立憑證」（或「建立用戶端」），然後選取「OAuth 用戶端 ID」。
2. 將應用程式類型設為「Desktop app」（電腦應用程式）。
3. 將其命名為 Workspace Agent Local。
4. 按一下「建立」並下載 JSON 檔案。將其儲存至本機，並命名為 client_secret.json。

4. 代理程式開發與測試

本節說明如何建構代理程式環境。我們會使用靜態權杖方法進行測試，也就是使用 `gcloud` 擷取 OAuth 權杖，並將其傳遞至 ADK 的 `McpToolset`。您可以使用本機電腦或 **Google Cloud Shell** 按照這些步驟操作。

選擇「本機」或「Google Cloud Shell」，設定開發環境。這兩種方法都會使用共用的 Python 指令碼進行驗證，確保各平台的一致性。

1. 設定環境目錄

在終端機中執行下列指令，建立專案目錄並安裝必要套件。

- **Google Cloud Shell**：首先，點選 Google Cloud 控制台右上角工具列中的「啟用 Cloud Shell」圖示 。
- **在本機電腦上**：開啟標準終端機。

```
mkdir -p gws-adk-agent/workspace_agent
cd gws-adk-agent
python3 -m venv .venv
source .venv/bin/activate
pip install google-adk poetry google-auth-oauthlib
```

重要事項：請確認終端機提示反映已啟用的環境 (應顯示 `(.venv)`)。如果您開啟新的終端機視窗或稍後重新連線至 Cloud Shell，請務必從 `gws-adk-agent` 目錄重新執行 `source .venv/bin/activate`，確保您使用的是正確版本的 ADK CLI。使用現有或全域環境可能會導致版本不符，以及網頁資產發生 404 錯誤。

2. 準備憑證檔案

您必須將上一步下載的 `client_secret.json` 檔案放在 `gws-adk-agent` 目錄的根目錄中。

- **本機**：將下載的 `client_secret.json` 檔案移動或複製到 `gws-adk-agent` 目錄。
- **Google Cloud Shell**：在 Cloud Shell 中執行下列指令來建立檔案 (將 `[PASTE_JSON_HERE]` 換成實際的 JSON 內容)：

```
cat << 'EOF' > client_secret.json
[PASTE_JSON_HERE]
EOF
```

3. 驗證並產生 `.env` (`auth.py`)

為確保驗證程序一致，並避免環境專屬的 CLI 路由問題，我們使用由官方 Google 驗證程式庫支援的自訂 Python 指令碼。這個指令碼會管理 OAuth 流程、在本機儲存應用程式預設憑證 (ADC)，並產生必要的 `.env` 檔案。

在 `gws-adk-agent` 目錄中建立名為 `auth.py` 的檔案，並加入下列程式碼：

```

import json
import os
from urllib.parse import urlparse, parse_qs
import google.auth
from google_auth_oauthlib.flow import InstalledAppFlow

CLIENT_SECRET_FILE = 'client_secret.json'

SCOPES = [
    "https://www.googleapis.com/auth/cloud-platform",
    "https://www.googleapis.com/auth/calendar",
    "https://www.googleapis.com/auth/chat.spaces.readonly",
    "https://www.googleapis.com/auth/chat.messages",
    "https://www.googleapis.com/auth/drive.readonly",
    "https://www.googleapis.com/auth/gmail.readonly",
    "https://www.googleapis.com/auth/gmail.compose",
    "https://www.googleapis.com/auth/directory.readonly",
    "https://www.googleapis.com/auth/contacts.readonly"
]

# Initialize the flow from the client secrets JSON
flow = InstalledAppFlow.from_client_secrets_file(
    CLIENT_SECRET_FILE,
    scopes=SCOPES,
    redirect_uri='http://localhost:8085/'
)

# Generate the Auth URL
auth_url, expected_state = flow.authorization_url(prompt='consent', access_type='offline')

print("\n=== GOOGLE OAUTH OFFICIAL LIBRARY FLOW ===")
print("1. Copy the following link and paste it into your browser (or click it if supported):\n")
print(auth_url)
print("\n2. Authorize the application.")
print("3. Your browser will redirect to a 'localhost' page (it will show a 'Site can't be reached' error, which is EXPECTED and normal).")
print("4. Copy the ENTIRE URL from your browser's address bar (including the http://localhost:8085/ part).\n")

# Get the redirected URL from the user
redirected_url = input("Paste the full localhost URL here: ").strip()

# Exchange the redirect URL for tokens
print("\nExchanging code for tokens...")
try:
    parsed_url = urlparse(redirected_url)
    query_params = parse_qs(parsed_url.query)

    returned_state = query_params.get('state', [None])[0]
    code = query_params.get('code', [None])[0]

```



```

    if not code:
        raise ValueError("No 'code' parameter found in the URL.")
    if returned_state != expected_state:
        raise ValueError("CSRF Warning! State mismatch.")

    flow.fetch_token(code=code)
    creds = flow.credentials
except Exception as e:
    print(f"Authentication failed: {e}")
    exit(1)

if not creds.refresh_token:
    print("\nError: No refresh token returned. You may need to revoke access and try again.")
    exit(1)

# Save Application Default Credentials
adc_data = {
    "client_id": creds.client_id,
    "client_secret": creds.client_secret,
    "refresh_token": creds.refresh_token,
    "type": "authorized_user"
}

adc_dir = os.path.expanduser("~/config/gcloud")
os.makedirs(adc_dir, exist_ok=True)
adc_path = os.path.join(adc_dir, "application_default_credentials.json")

with open(adc_path, "w") as f:
    json.dump(adc_data, f, indent=2)

# Detect Project ID
try:
    _, project_id = google.auth.default()
except Exception:
    project_id = None

project_id = project_id or os.environ.get("GOOGLE_CLOUD_PROJECT", "YOUR_PROJECT_ID")

# Save to .env for local development (relative to project root)
env_dir = "workspace_agent"
os.makedirs(env_dir, exist_ok=True)
env_path = os.path.join(env_dir, ".env")

with open(env_path, "w") as f:
    f.write("GOOGLE_GENAI_USE_VERTEXAI=1\n")
    f.write(f"GOOGLE_CLOUD_PROJECT={project_id}\n")
    f.write("GOOGLE_CLOUD_LOCATION=us-central1\n")

print(f"\nSuccess! Application Default Credentials saved to: {adc_path}")
print(f"Environment variables saved to: {env_path}")
if project_id == "YOUR_PROJECT_ID":

```

```
print("NOTE: Could not automatically detect Project ID. Please update it manually in  
.env")
```

在終端機中執行指令碼：

```
python3 auth.py
```

建立代理程式碼

無論您在上一個步驟中選擇哪個環境，服務專員程式碼都完全相同。在預先建立的 `workspace_agent` 子目錄中，建立名為 `agent.py` 的檔案，並加入下列程式碼。這個指令碼會使用應用程式預設憑證 (ADC) 提供授權，並透過動態標題供應器，在記憶體中自動驗證及重新整理憑證。

```

import os
import datetime

import google.auth
from google.auth.transport.requests import Request
from google.adk.agents.llm_agent import LlmAgent
from google.adk.tools.mcp_tool.mcp_toolset import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import StreamableHTTPConnectionParams

# Enable graceful error handling so timeouts return as tool outputs instead of crashing
os.environ["ADK_ENABLE_MCP_GRACEFUL_ERROR_HANDLING"] = "1"

MODEL = "gemini-2.5-flash"

# Inject User ADC path globally so both ADK and underlying GenAI SDKs
# load it safely, bypassing broken Compute Engine metadata fallbacks.
_adc_path = os.path.expanduser(
    "~/config/gcloud/application_default_credentials.json"
)
if not os.path.exists(_adc_path):
    raise FileNotFoundError(
        f"User ADC not found at {_adc_path}. "
        "Please run 'python3 auth.py' first to generate credentials."
    )
os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = _adc_path

creds, _ = google.auth.default()

class TokenRelay:
    """Manages in-memory caching of Google OAuth access tokens.

    Decouples token refresh logic from credentials validation states to
    prevent infinite refresh loops during automated tool retries.
    """

    def __init__(self, credentials):
        self._creds = credentials
        self._cached_token: str | None = None
        self._expiry: datetime.datetime | None = None

    def get_token(self) -> str:
        now = datetime.datetime.now()
        # Refresh if no cache or expiring within 5 minutes
        if (
            self._cached_token is None
            or self._expiry is None
            or now >= (self._expiry - datetime.timedelta(minutes=5))
        ):
            self._creds.refresh(Request())
            self._cached_token = self._creds.token
            # Token lives for 60 mins; cache for 50 mins safely

```

```

        self._expiry = now + datetime.timedelta(minutes=50)
        print("\n[Agent Runtime] Access token refreshed from ADC.")
        return self._cached_token or ""

# Initialize auth relay and perform startup safety check
token_relay = TokenRelay(creds)
token_relay.get_token()

def auth_header_provider(tool_context=None) -> dict[str, str]:
    return {"Authorization": f"Bearer {token_relay.get_token()}"}}

# Initialize the 5 GWS MCP servers with dynamic runtime provider
calendar_mcp = McpToolset(
    connection_params=StreamableHTTPConnectionParams(
        url="https://calendarmcp.googleapis.com/mcp/v1",
    ),
    header_provider=auth_header_provider
)
chat_mcp = McpToolset(
    connection_params=StreamableHTTPConnectionParams(
        url="https://chatmcp.googleapis.com/mcp/v1",
    ),
    header_provider=auth_header_provider
)
drive_mcp = McpToolset(
    connection_params=StreamableHTTPConnectionParams(
        url="https://drivemcp.googleapis.com/mcp/v1",
    ),
    header_provider=auth_header_provider
)
gmail_mcp = McpToolset(
    connection_params=StreamableHTTPConnectionParams(
        url="https://gmailmcp.googleapis.com/mcp/v1",
    ),
    header_provider=auth_header_provider
)
people_mcp = McpToolset(
    connection_params=StreamableHTTPConnectionParams(
        url="https://people.googleapis.com/mcp/v1",
    ),
    header_provider=auth_header_provider
)

current_date = datetime.datetime.now().strftime("%Y-%m-%d")

# Define the agent and attach all 5 toolsets
root_agent = LlmAgent(
    model=MODEL,
    name='gws_adk_agent',

```

```
instruction=f"""You are a helpful assistant grounded in the user's Google Workspace data.
Today's current date is {current_date}. Always calculate relative dates (like 'this week'
or 'upcoming meetings') using this reference.

Use the provided MCP tools to answer questions about their Calendar, Chat, Drive, Gmail,
and Contacts.

If a tool returns an error indicating a timeout, inform the user that the service took
too long to respond."""
tools=[calendar_mcp, chat_mcp, drive_mcp, gmail_mcp, people_mcp]
)
```

執行及測試代理

繼續操作前：確認虛擬環境仍處於啟用狀態（終端機提示中應會顯示 `(.venv)`）。如果未啟用，請從 `gws-adk-agent` 目錄執行 `source .venv/bin/activate`。

您可以透過互動式網頁 UI 或直接在終端機/殼層中與代理互動。

選項 1：互動式網頁 UI

如果您使用本機電腦：

1. 從 `gws-adk-agent` 專案根目錄啟動 ADK 網頁介面：

```
adk web
```

2. 在瀏覽器中開啟 `http://localhost:8000`，即可與服務專員交談。

如果您使用 Google Cloud Shell：

1. 從 `gws-adk-agent` 專案根目錄啟動 ADK 網頁介面，明確強制使用通訊埠 8080：

```
adk web --port 8080 --allow_origins=*
```

2. 按一下 Cloud Shell 工具列右上方的「網頁預覽」按鈕，然後選取「透過以下通訊埠預覽：8080」，即可在新分頁中開啟介面。

選項 2：終端機 CLI 模式 (替代方案)

如果您偏好使用終端機，或想避免設定網路瀏覽器預覽畫面，ADK 提供原生互動式 REPL 對話模式。

在 `gws-adk-agent` 專案根目錄中執行下列指令（這在本機電腦和 Google Cloud Shell 上的運作方式相同）：

```
adk run workspace_agent
```

系統會直接在殼層中開啟互動式即時通訊工作階段：

```
Running agent gws_adk_agent, type exit to exit.
```

```
[user]:
```

提示代理程式

無論選擇哪個介面，請嘗試使用 Workspace MCP 工具的查詢測試代理程式：

What are my upcoming meetings this week?

Summarize the last 3 unread emails in my Gmail.

步驟 5 · 約 20 分鐘

5. 正式環境部署

如要將代理程式部署至正式環境，我們無法使用硬式編碼的本機權杖。我們不會這麼做，而是使用 ADK 的 `header_provider`，在使用者與代理互動時，動態擷取 Gemini Enterprise 平台插入的 OAuth 存取權杖。

建立正式版代理程式碼

建立名為 `enterprise_ai` 的目錄套件，並在其中建立 `agent.py` 檔案：

```
mkdir -p enterprise_ai
```

將下列內容寫入 `enterprise_ai/agent.py`：

```

import os
import datetime
import re
from google.adk.agents.llm_agent import LlmAgent
from google.adk.tools.mcp_tool.mcp_toolset import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import StreamableHTTPConnectionParams
from google.adk.tools.tool_context import ToolContext

# Enable graceful error handling so timeouts return as tool outputs instead of crashing
os.environ["ADK_ENABLE_MCP_GRACEFUL_ERROR_HANDLING"] = "1"

MODEL = "gemini-2.5-flash"
# This name MUST match the Authorization Name used during Gemini Enterprise registration
CLIENT_AUTH_NAME = "workspace-adk-auth"

def _get_access_token_from_context(tool_context: ToolContext) -> str:
    """Dynamically parses the user bearer token injected into the ToolContext state."""
    escaped_name = re.escape(CLIENT_AUTH_NAME)
    pattern = re.compile(fr"^{escaped_name}_\d+$")
    state_dict = tool_context.state.to_dict() if hasattr(tool_context.state, 'to_dict') else
tool_context.state
    matching_keys = [k for k in state_dict.keys() if pattern.match(k)]
    if matching_keys:
        return state_dict.get(matching_keys[0])
    raise Exception(f"No bearer token found in ToolContext state matching pattern
{pattern.pattern}")

def auth_header_provider(tool_context: ToolContext) -> dict[str, str]:
    """Provides the dynamic Authorization header for MCP requests."""
    token = _get_access_token_from_context(tool_context)
    return {"Authorization": f"Bearer {token}"}

# Initialize toolsets using the dynamic header_provider
calendar_mcp = McpToolset(

connection_params=StreamableHTTPConnectionParams(url="https://calendarmcp.googleapis.com/mcp/
v1"),
    header_provider=auth_header_provider
)
chat_mcp = McpToolset(

connection_params=StreamableHTTPConnectionParams(url="https://chatmcp.googleapis.com/mcp/v1")
,
    header_provider=auth_header_provider
)
drive_mcp = McpToolset(

connection_params=StreamableHTTPConnectionParams(url="https://drivemcp.googleapis.com/mcp/v1"
),
    header_provider=auth_header_provider
)
gmail_mcp = McpToolset(

```



```

connection_params=StreamableHTTPConnectionParams(url="https://gmailmcp.googleapis.com/mcp/v1"
),
    header_provider=auth_header_provider
)
people_mcp = McpToolset(

connection_params=StreamableHTTPConnectionParams(url="https://people.googleapis.com/mcp/v1"),
    header_provider=auth_header_provider
)

current_date = datetime.datetime.now().strftime("%Y-%m-%d")

root_agent = LlmAgent(
    model=MODEL,
    name='enterprise_ai',
    instruction=f""""You are an enterprise assistant grounded securely in the user's Workspace
data.
    Today's current date is {current_date}. Always calculate relative dates (like 'this week'
or 'upcoming meetings') using this reference.
    Always use the provided MCP tools to fetch context from Calendar, Chat, Drive, Gmail, and
People.
    If a tool returns an error indicating a timeout, inform the user that the service took
too long to respond.""",
    tools=[calendar_mcp, chat_mcp, drive_mcp, gmail_mcp, people_mcp]
)

```

部署至 GEAP Agent Runtime

使用 ADK CLI 部署專業程式碼代理：

```

adk deploy agent_engine \
  --project=$(gcloud config get-value project) \
  --region=us-central1 \
  --display_name="Workspace ADK Agent" \
  enterprise_ai

```

等待部署完成，然後從終端機輸出內容複製產生的 **Reasoning Engine 資源名稱** (例如 `projects/PROJECT_ID/locations/us-central1/reasoningEngines/ENGINE_ID`)。

6. 在 Gemini Enterprise 中註冊

現在，我們將自訂 ADK 代理帶進 **Gemini Enterprise**，使用者可以直接與其對話。

建立正式版用戶端 ID (網頁應用程式)

如要將代理程式部署至正式環境，請務必建立網頁應用程式用戶端 ID。與用於本機測試的電腦版用戶端不同，網頁應用程式用戶端可啟用安全的伺服器端 OAuth 流程。這樣一來，GEAP Agent Runtime 上的代管代理程式就能使用指定的重新導向 URI，安全地接收 Gemini Enterprise 轉送的使用者驗證權杖。這項設定是必要的，因為代理程式必須在正式環境中，代表使用者安全地存取 Google Workspace 資料。

1. 在 Google Cloud 控制台中，按一下左上角的「導覽選單」(漢堡圖示)。依序選取「Google Auth Platform」>「Clients」(如果沒有看到「Google Auth Platform」，請依序選取「APIs & Services」>「Credentials」)。按一下「建立憑證」(或「建立用戶端」)，然後選取「OAuth 用戶端 ID」。
2. 將應用程式類型設為「Web application」(網頁應用程式)。
3. 將其命名為 `Workspace Agent Production`。
4. 在「已授權的重新導向 URI」部分，新增下列 URI：
 - 按一下「新增 URI」，然後輸入 `https://vertexaisearch.cloud.google.com/oauth-redirect`。
 - 再次點選「新增 URI」，然後輸入 `https://vertexaisearch.cloud.google.com/static/oauth/oauth.html`。
5. 按一下「建立」，在「已建立 OAuth 用戶端」彈出式視窗(或「用戶端」清單)中，複製「用戶端 ID」和「用戶端密鑰」。在 Gemini Enterprise 中註冊代理時，您需要這些資訊。

建立 Gemini Enterprise 應用程式

1. 前往 Google Cloud 控制台的「Gemini Enterprise」頁面：
 1. 在「Gemini Enterprise」資訊卡下方，點選「管理」。
 2. 點選「Create app」(建立應用程式)。
 3. 在「Gemini Enterprise」部分，點選「建立」。
 4. 在「應用程式名稱」欄位，輸入應用程式名稱。應用程式 ID 會顯示在應用程式名稱下方。
 5. 在「External name of your company or organization」(貴公司或貴機構的外部名稱)欄位中，輸入公司或機構名稱。在本教學課程中，您可以使用 `Cymbal Bank`。
 6. 選取「global (Global)」(全球 (全球)) 做為應用程式的位置。
 7. 按一下「繼續」。

註冊自訂代理

1. 開啟 **Gemini Enterprise** 控制台：
 1. 選取有效應用程式，前往「Agents」，然後依序點選「+ Add agent」>「Add Custom agent via Agent Runtime」。
2. 在「授權」部分，按一下「新增授權」：
 - 授權名稱：`workspace-adk-auth` (必須與 *Python* 程式碼中的 `CLIENT_AUTH_NAME` 完全相符)
 - 用戶端 ID：貼上上方建立的「正式版網頁應用程式」用戶端 ID。
 - 用戶端密鑰：貼上上方建立的「Production Web Application」(正式版網路應用程式) 用戶端密鑰。
 - 權杖 URI：`https://oauth2.googleapis.com/token`

- **授權 URI**：將 `YOUR_CLIENT_ID` 替換為您在上述步驟 5 複製的用戶端 ID，即可建構 URI：

```
https://accounts.google.com/o/oauth2/v2/auth?
client_id=YOUR_CLIENT_ID&redirect_uri=https%3A%2F%2Fvertexaisearch.cloud.google.com%2F
static%2Foauth%2Foauth.html&scope=https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcalendar.r
eadonly%20https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcalendar.calendars%20https%3A%2F%2
Fwww.googleapis.com%2Fauth%2Fcalendar.events%20https%3A%2F%2Fwww.googleapis.com%2Fauth
%2Fcloud-
platform%20https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fgmail.send%20https%3A%2F%2Fwww.go
ogleapis.com%2Fauth%2Fgmail.readonly%20https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fchat.
spaces%20https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fchat.messages%20https%3A%2F%2Fwww.g
oogleapis.com%2Fauth%2Fdrive.readonly%20https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fdriv
e.file%20https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fdirectory.readonly&include_granted_
scopes=true&response_type=code&access_type=offline&prompt=consent
```

3. 依序點選「完成」和「下一步」。

4. 在「設定」部分：

- **代理程式名稱**：`Workspace Pro Agent`
- **說明**：

```
Use this agent to answer questions about your Google Workspace data, including Gmail,
Google Drive, Google Calendar, Google Chat, and Contacts. It can find files, summarize
emails, check your schedule, and look up contact information.
```

- **Agent Runtime 推論引擎**：貼上從 `adk deploy` 輸出內容複製的引擎資源名稱。

5. 按一下「建立」。

在 Gemini Enterprise 中試用 Agent

您可以使用 **Google Cloud 控制台搶先版** (開發人員最快) 或 **Gemini Enterprise 網頁應用程式** (使用者體驗) 與代理程式互動。

選項 1：Google Cloud 控制台預覽版

1. 前往 Google Cloud 控制台的「Gemini Enterprise」頁面：
1. 在導覽選單中，按一下「應用程式」，然後選取您剛建立的應用程式。
2. 按一下右上方的「預覽」或「開啟預覽」。
3. 在搜尋列中輸入下列查詢：

```
Search for files in Drive related to 'Project Milestone', summarize them, and tell me if
I have any meetings with the project owner today.
```

4. 按下 **Enter** 鍵提交查詢，並直接在控制台中查看結果。

選項 2：Gemini Enterprise 網頁應用程式

1. 開啟 Gemini Enterprise 網頁應用程式介面。
2. 依序前往「選單」圖示 ≡ > 「代理程式」，然後選取「Workspace Pro 代理程式」下方的「來自貴機構」。

3. 輸入與 Google Cloud 控制台「預覽」選項相同的查詢。
 4. 系統提示您完成 OAuth 使用者同意聲明流程時，請按一下「授權」。
 5. 代理程式會使用 MCP 通訊協定，在多項服務中順暢執行工作！
-

步驟 7 · 約 2 分鐘

7. 清除所用資源

為避免產生不必要的費用，請清理資源：

主控台選項

前往 [API 和服務資訊主頁](#)，選取已啟用的 MCP 服務 (例如 Calendar MCP、Gmail MCP)，然後按一下「停用 API」。刪除「Google Auth Platform」>「用戶端」下的 OAuth 用戶端 ID，並從 **Gemini Enterprise 控制台** 刪除 Reasoning Engine 部署作業。

CLI 選項

執行下列終端機指令，停用 MCP 服務：

```
# Disable Workspace MCP services
gcloud services disable calendarmcp.googleapis.com \
  chatmcp.googleapis.com \
  drivemcp.googleapis.com \
  gmailmcp.googleapis.com
```

步驟 8 · 約 1 分鐘

8. 恭喜

恭喜！您已成功建構、測試及部署直接連線至官方 Google Workspace MCP 伺服器的專業程式碼 ADK AI 代理程式。

目前所學內容

- 如何在 Google Cloud 中啟用 Google Workspace MCP 服務。
- 如何使用靜態標頭處理本機 ADK 測試的 OAuth 憑證。
- 如何使用 `header_provider` 為正式版代理程式導入動態權杖插入功能。
- 如何將 ADK 代理部署至 GEAP Agent Runtime，並在 Gemini Enterprise 中註冊。

參考文件

- [Model Context Protocol \(MCP\) 規格](#)
 - [ADK 說明文件 \(專業程式碼架構\)](#)
 - [設定 Google Workspace MCP 伺服器](#)
-